

Backend Developer Take-Home Assignment

Project Overview: "SaatCMS" Dynamic OTT Middleware Core

Context

At **Saat Teknoloji**, our product development team focuses heavily on the OTT Content Management System (CMS) and Middleware (MW) API layers. One of our primary challenges is managing complex content hierarchies (Movies, Series, Seasons, Episodes, Live Channels) and dynamically applying business rules (licensing, device management, and EPG scheduling) before delivering the metadata payload to client applications (Smart TV, Mobile, Web).

Your task is to build a prototype of the **SaatCMS Middleware Core Engines**. While Java Spring Boot is preferred and highly recommended by our stack, you are welcome to use any tech stack you feel most comfortable with, provided it fulfills all technical specifications.

Important Note: This assignment evaluates your practical understanding of data modeling, data integrity under concurrent requests, and inheritance logic beyond standard, simple CRUD APIs.

Core Requirements

1. Hierarchical Content Metadata & Rule Inheritance Engine

An OTT CMS must avoid redundant data entry. You need to implement a dynamic metadata inheritance system.

- Create an endpoint: `GET /api/v1/mw/content/{contentId}`.
- The content hierarchy must support at least: **Series -> Season -> Episode**.
- **The Override Logic:** A `Series` has default properties (e.g., `parentalRating`, `genre`, and `geoBlockCountries`). A `Season` or an individual `Episode` can *optionally* override any of these properties.
- When the middleware endpoint is called for an `Episode`, your code must dynamically traverse up the tree to construct the final payload. If the `Episode` has no specific `parentalRating`, it takes it from the `Season`. If the `Season` doesn't have it, it takes it from the `Series`.

2. Live EPG (Electronic Program Guide) Overlap Validator

The CMS allows operators to schedule live programs on TV channels. You must write a strict validator within your CMS API to prevent human errors during data entry.

- Create an endpoint: `POST /api/v1/cms/channels/{channelId}/epg`.
- The payload will contain: `programName`, `startTime`, and `endTime`.

- **The Rule:** The system must strictly reject (400 Bad Request) any new program that overlaps with an existing schedule on that specific channel.
- *Concurrency Constraints:* EPG data often handles timezone offsets. Your validator must handle inputs in UTC and ensure thread-safe validation so that two concurrent operator requests cannot accidentally create overlapping programs for the same channel.

3. Middleware Entitlement & Device Gatekeeper

Before the Middleware returns a playback URL or asset details to a user, it must validate complex licensing and device rules.

- Create an endpoint: `GET /api/v1/mw/playback/{contentId}` .
- The endpoint must accept headers: `X-User-Id` , `X-User-Country` , and `X-Device-Type` .
- You must implement an internal authorization pipeline that checks:
 1. **Geofencing:** Does the constructed metadata (from Requirement 1) allow streaming in the user's country?
 2. **Device Rules:** Certain premium assets (e.g., 4K Movies) are restricted and can *only* be requested from `SmartTV` or `Web` , but strictly blocked on `Mobile` .
- If any rule fails, return `403 Forbidden` with a specific OTT error code inside a JSON body (e.g., `{"errorCode": "GEO_BLOCKED"}` or `{"errorCode": "DEVICE_NOT_SUPPORTED"}`).

Technical Constraints & Stack

- **Tech Stack:** Java Spring Boot 3.x is highly preferred. However, you are free to use any modern backend framework (Node.js, Go, .NET, Python, etc.) that you excel in.
- **Database:** Use an embedded or lightweight storage mechanism (e.g., H2, SQLite, or PostgreSQL via Docker). Please provide seed data or scripts to populate a sample Series hierarchy and a Live Channel.
- **No External Validation Libraries:** The EPG overlap logic and metadata inheritance must be implemented natively within your custom application logic.

Evaluation Criteria

Criteria	Description
Domain Logic Accuracy	How effectively you handle the metadata inheritance tree without causing performance bottlenecks or database N+1 issues.
Algorithmic Correctness	The robustness of the EPG overlap validation algorithm under edge cases (e.g., a program starting exactly when another ends).
Data Integrity & Concurrency	Ensuring that concurrent API requests or race conditions cannot bypass the entitlement or EPG rules.
Code Quality & Architecture	Clean separation of concerns, implementation of appropriate design patterns, error handling, and coverage of unit/integration tests.
Documentation	A clear <code>README.md</code> explaining how to run the project, how to test it, and architectural choices.

Submission Guidelines

- Please commit your code to a public/private GitHub repository and share the link with us.
- Provide a collection of Postman requests or a list of cURL commands in the `README.md` that demonstrates successful responses and specific failure cases (overlap blocked, geo-blocked, device-blocked).
- **Deadline:** 7 days from today.

Good Luck!

Saat Teknoloji Product Development Team